Mobile API changes — 2026-11-11 (for the Flutter app)

These are **breaking changes**. The app must be updated before this backend version is deployed, or shipped together with it. Everything below is under the base path `/api/v1`.

Where this document disagrees with older documents (`mobile-api-specification.md`, `flutter-api-integration-sequence.md`, `mobile-flutter-media-quiz-integration-guide.md`, earlier `mobile-api-changes-*.md`), this one is right.

Summary of what to change in the app

#	Change	Effort
1	Ads are earned through **one flow** only: `/ad-views/*`. Three older ways are removed.	Rewrite the ad / quiz / survey-on-ad screens
2	Quiz and survey answers of an ad go to `POST /ad-views/{adViewId}/answers`.	Change one call
3	Every `/mobile/**` response is now wrapped in the standard envelope. Surveys and quizzes were bare before.	Change parsing for 8 endpoints
4	Every paged list has one fixed shape. `pageable` and `sort` are gone.	One shared page model
5	Two duplicate endpoints are removed.	Change one path
6	A reward can now be **held for review** instead of paid at once.	Show "pending" state
7	Timestamp rule is now written down (no format change).	Check date parsing

---

1. One ad flow

Removed — stop calling these

Removed	Use instead
`GET /mobile/ads/feed`	`GET /mobile/content/ads` (also `/mobile/content/quizzes`, `/mobile/content/surveys`)
`POST /mobile/ads/{id}/view`	`POST /ad-views/start`, then `POST /ad-views/{adViewId}/complete`

Removed	Use instead
`POST /mobile/campaigns/{id}/start`	`POST /ad-views/start` with the campaign's ad id
`POST /mobile/campaigns/{id}/complete`	`POST /ad-views/{adViewId}/complete`
`POST /quizzes/submit`	`POST /ad-views/{adViewId}/answers`
`POST /surveys/submit`	`POST /ad-views/{adViewId}/answers`
`GET /feed/recommended`	`GET /mobile/home-feed` (already ranked)

A removed endpoint answers **404** (`errorCode: "NOT_FOUND"`).

The flow

```
1. List        GET  /mobile/content/ads?page=0&size=20        (or /content/quizzes, /content/surveys)
               GET  /mobile/content/{id}                       one item with its questions
               GET  /mobile/campaigns/{campaignId}/items       every item of one campaign
2. Start       POST /ad-views/start                            when the media starts playing
3. Complete    POST /ad-views/{adViewId}/complete              when the media has been watched
4. Answers     POST /ad-views/{adViewId}/answers               only when the item has questions
5. Click       POST /ad-views/{adViewId}/click                 when the user opens the ad's link (optional)
```

Items. Each list item (`MobileContentItemDto`) has `id` (this is the **ad id** for step 2), `kind` (`AD` , `QUIZ` , `SURVEY`), `title` , `description` , `mediaType` , `mediaUrl` , `thumbnailUrl` , `durationSeconds` , `reward` , `paysPerQuestion` , `questionCount` , `passingScore` , `progress` , `available` , `unavailableCode` , `unavailableMessage` , `campaignId` , `campaignTitle` , `campaignBannerUrl` , `campaignEndAt` , `destinationUrl` . Do not start an item whose `available` is `false` ; show `unavailableMessage` .

2. Start

```
POST /api/v1/ad-views/start
{ "adId": "<item id>", "deviceId": "<stable device id>" }
```

```
{ "success": true, "message": "Ad view started",
  "data": { "adViewId": "…", "startedAt": "2026-11-11T14:30:00" } }
```

Keep `adViewId`; every later step uses it. The server measures watch time from this call, so call it when playback really begins.

3. Complete

```
POST /api/v1/ad-views/{adViewId}/complete
{ "watchedDurationSeconds": 30 }
```

```
{ "success": true, "message": "Ad completed",
  "data": { "id": "…", "adId": "…", "status": "REWARDED", "watchedDurationSeconds": 30,
            "rewardEligible": true, "rewardEarned": 2.50, "startedAt": "…",
"completedAt": "…",
            "questionMode": null, "awaitingAnswers": false } }
```

- The user must have watched at least **80 %** of `durationSeconds`, and that much time must really have passed since step 2. Otherwise: `ILLEGAL_STATE` "Insufficient watch duration…".
- `status` is `REWARDED` for a plain ad (kind `AD`): the reward is settled now (see section 6).
- `status` is `COMPLETED` with `awaitingAnswers: true` for an item with questions: nothing is paid yet; go to step 4.

4. Answers (kind `QUIZ` or `SURVEY`, or an `AD` with follow-up questions)

```
POST /api/v1/ad-views/{adViewId}/answers
{ "answers": [
    { "questionId": "…", "optionId": "…" },
    { "questionId": "…", "textAnswer": "free text" }
] }
```

```
{ "success": true, "message": "Answers submitted",
  "data": { "passed": true, "score": 3, "wrongAnswers": 0, "totalQuestions": 3,
"pointsEarned": 3,
            "rewardTriggered": true, "rewardEarned": 5.00, "cashbackPromised": 0,
            "participationId": "…", "participationReference": "…", "cashback": [] } }
```

The body no longer carries `adId`; the ad view identifies the ad. The view must be the caller's own and must be completed first. A failed quiz answers `200` with `passed: false` and `rewardEarned: 0`.

5. Click

```
POST /api/v1/ad-views/{adViewId}/click        (no body)
```

```
{ "success": true, "data": { "adViewId": "…", "clickedAt": "…", "destinationUrl":
"https://…", "firstClick": true } }
```

Open `destinationUrl`. Calling it again is harmless (`firstClick: false`).

Errors in this flow

HTTP	`errorCode`	Meaning
409	`ALREADY_CLAIMED`	The user was already rewarded for this ad
409	`CAMPAIGN_INACTIVE`, `CAMPAIGN_NOT_STARTED`, `CAMPAIGN_ENDED`, `OUTSIDE_DELIVERY_SCHEDULE`	Campaign is not running now
409	`CAMPAIGN_BUDGET_EXHAUSTED`, `PARTICIPANT_TARGET_REACHED`, `DAILY_VIEW_CAP_REACHED`	Nothing left to earn
403	`NOT_IN_TARGET_AUDIENCE`	Not for this user
409	`DUPLICATE_PARTICIPATION`	Already took part
400	`INVALID_ANSWER`	An answer does not fit its question
500	`ILLEGAL_STATE`	Rule broken: watch time too short, view already completed, view not the caller's, daily limit of 100

Rely on `errorCode`, never on the message text.

---

2. One response envelope

Every successful response of `/mobile/**` and `/ad-views/**` is:

```
{ "success": true, "message": "…", "data": <payload>, "timestamp": "2026-11-11T14:30:00"
}
```

`data` is left out when there is no payload. Every error is:

```
{ "success": false, "errorCode": "…", "message": "…", "timestamp": "…" }
```

Endpoints that were bare and are now wrapped

Endpoint	Before	Now
`GET /mobile/surveys`	`[ … ]`	`{ success, data: [ … ] }`
`GET /mobile/surveys/{id}`	`{ … }`	`{ success, data: { … } }`
`POST /mobile/surveys/{id}/heartbeat`	`{ … }`	`{ success, data: { … } }`
`POST /mobile/surveys/{id}/discard`	`{ success, message }`	`{ success, message }` (no `data`)
`POST /mobile/surveys/{id}/submit`	`{ … }`	`{ success, data: { … } }`
`GET /mobile/quizzes`	`[ … ]`	`{ success, data: [ … ] }`
`GET /mobile/quizzes/{id}`	`{ … }`	`{ success, data: { … } }`
`POST /mobile/quizzes/{id}/submit`	`{ … }`	`{ success, data: { … } }`

The payloads inside `data` are unchanged. `/mobile/surveys` and `/mobile/quizzes` are the **standalone** surveys and quizzes; they are not part of the ad flow and keep their own submit endpoints.

Suggested model: one generic `ApiResponse<T>` with `fromJson(json, T Function(Object?) fromData)`; throw an `ApiException(errorCode, message)` when `success` is `false`.

---

3. One page shape

Every paged list answers with this in `data`:

```
{ "content": [ … ],
  "number": 0, "size": 20,
  "totalElements": 57, "totalPages": 3, "numberOfElements": 20,
  "first": true, "last": false, "hasNext": true, "empty": false }
```

`pageable`, `sort` and other Spring fields **no longer exist**. If the app read `data.pageable.pageNumber`, read `data.number`. Request parameters are unchanged: `page` (from 0) and `size`.

Paged endpoints: `/mobile/content/ads`, `/mobile/content/quizzes`, `/mobile/content/surveys`, `/mobile/campaigns`, `/mobile/campaigns/participations`, `/mobile/campaigns/cashback`, `/mobile/notifications`, `/mobile/wallet/transactions`, `/mobile/withdrawals`, `/mobile/rewards/history`.

Not paged (a plain array in `data`): `/mobile/surveys`, `/mobile/quizzes`, `/mobile/social-tasks`, `/mobile/referrals/history`, `/mobile/withdrawals/accounts`, `/mobile/payment-methods`, `/mobile/banners`, `/mobile/campaigns/featured`, `/mobile/campaigns/trending`, `/mobile/campaigns/categories`.

---

4. Duplicates removed

Removed	Use instead
`GET /mobile/notification-preferences`	`GET /mobile/settings/notifications`
`PUT /mobile/notification-preferences`	`PUT /mobile/settings/notifications`

Same request and response body.

---

5. Rewards can be held for review

Before, a completed ad was paid at once unless the user's behaviour score was high. Now two checks run when a reward is earned (ad completion or answers):

1. the behaviour score, as before, and
2. the fraud team's risk assessment, using the request's real IP address, device id and user agent.

If either objects, the reward is **held**: it sits in the pending balance until an admin approves or rejects it. The API response looks the same either way (`status: "REWARDED"`, `rewardEarned` set), so:

- After a reward, refresh `GET /mobile/wallet/overview`. A paid reward raises `availableBalance`; a held one raises `pendingBalance`.
- `GET /mobile/rewards/history` shows each reward as `PENDING`, `APPROVED` or `REJECTED`.
- Show held rewards as "under review", not as a failure.

Send a stable `deviceId` in `POST /ad-views/start` (the same value for the same install). A device shared by many accounts is one of the things the assessment looks at.

Incentive bonuses

When a reward is paid, a running incentive may pay an extra bonus on top (set up by the finance team, with limits per day and per user). It arrives a moment later as a separate wallet transaction of type `BONUS` with

the remark "Incentive: ". Nothing to call; it appears in `/mobile/wallet/transactions` and in today's earnings. Do not assume the wallet changes by exactly `rewardEarned`.

---

6. Values that are now real

Endpoint	Field	Before	Now
`GET /mobile/profile/stats`	`currentStreak`	always `0`	the real streak, same as `/mobile/home-feed` and `/mobile/progress`
`GET /mobile/profile/stats`	`campaignsCompleted`	counted campaign sessions	campaigns the user has been rewarded in
`GET /mobile/config/static`	`countries`	always `["Bangladesh"]`	active countries from the server's geo data
`GET /mobile/tasks/daily`	"Complete 2 Campaigns" progress	counted campaign sessions	campaigns rewarded today

---

7. Timestamps

No format change; this is the rule to code against.

Kind	Example	Meaning
Without offset (almost every field, and `timestamp` in the envelope)	`2026-11-11T14:30:00`	**Asia/Dhaka** wall-clock time (UTC+06:00). Parse it as Dhaka time, not as device-local time and not as UTC.
With `Z` (campaign `startAt`, `endAt`, `campaignEndAt`)	`2026-11-11T08:30:00Z`	UTC. Convert to the device's zone for display.

`GET /mobile/sync?since=` takes a value **without offset**, in Dhaka time. The safest value is the `timestamp` of the last successful sync response, sent back unchanged.

---

8. Not changed

Auth (`/auth/**`), profile, KYC, wallet, withdrawals, notifications, referrals, social tasks, banners, home feed, app config, security and device endpoints keep their paths and payloads. Only the page shape (section 3) applies to their paged lists.

Commerce has no `/mobile` endpoints; the app uses the store API (`/commerce/store/**`) directly, which already uses the same envelope.

Push notifications: the server can now send real pushes through Firebase Cloud Messaging once it is switched on in production. The app keeps registering its token with `POST /mobile/notifications/register-token` ; nothing changes in the call.

Checklist

- [ ] Remove calls to the seven endpoints in section 1 and the two in section 4.
- [ ] Ad screen: `start` on play, `complete` at the end, `answers` when `awaitingAnswers` is `true` .
- [ ] Wrap surveys and quizzes parsing in the envelope.
- [ ] Replace the page model; remove every use of `pageable` .
- [ ] Handle errors by `errorCode` .
- [ ] After a reward, refresh the wallet and show pending rewards as "under review".
- [ ] Send a stable `deviceId` .
- [ ] Parse offset-less timestamps as Asia/Dhaka.